Курс по сетям для программистов

IPv6 в современной инфраструктуре

Адреса, DNS, URL, сокеты и диагностика приложений

Зачем программисту IPv6

- IPv6 влияет не только на сетевую инфраструктуру, но и на работу приложений: DNS, URL, bind-адреса, localhost, контейнеры и диагностику сетевых ошибок.
- Приложение может получить IPv6-адрес через DNS AAAA.
- localhost может разрешиться в ::1, а не в 127.0.0.1.
- Сервер может слушать IPv4, IPv6 или оба стека.
- IPv6-адрес в URL записывается в квадратных скобках.
- Ошибки IPv6 часто выглядят как «сервер не отвечает», хотя проблема в DNS, маршруте или bind-адресе.

Учебные вопросы

- 1. Где программист встречает IPv6.
- 2. Как читать IPv6-адрес и prefix.
- 3. Какие IPv6-адреса важны приложению.
- 4. IPv6 в URL, localhost и bind-адресах.
- 5. DNS A/AAAA и dual stack.
- 6. Маршруты, gateway и SLAAC на уровне приложения.
- 7. Диагностика IPv6-проблем в backend/frontend-сценариях.

1. Где программист встречает IPv6

IPv6 как часть работы приложения

IPv6 уже может быть включён

- Windows, Linux, macOS и мобильные ОС обычно поддерживают IPv6 по умолчанию.
- Клиент может получить IPv6-адрес автоматически.
- DNS может вернуть не только A, но и AAAA-запись.
- HTTP-клиент, браузер или runtime могут попробовать IPv6 раньше IPv4.
- Приложение может вести себя по-разному в IPv4-only, IPv6-only и dual stack средах.

Типовые места, где IPv6 ломает ожидания

- Локальная разработка: localhost возвращает ::1.
- Backend слушает 127.0.0.1, а клиент подключается к ::1.
- Сервис опубликован по AAAA, но firewall не разрешает IPv6.
- Контейнер или VM имеет IPv4-связь, но не имеет IPv6-маршрута.
- В логах указан IPv6-адрес клиента, и парсер ожидает IPv4.
- В конфиге URL указан IPv6-адрес без квадратных скобок.

Взгляд администратора и программиста

Администратор

- Планирует prefix.
- Настраивает gateway.
- Публикует AAAA.
- Фильтрует IPv6-трафик.
- Диагностирует сеть.

Программист

- Пишет URL и конфиги.
- Выбирает bind-адрес.
- Проверяет A/AAAA.
- Отлаживает connect timeout.
- Читает IP в логах.

Путь запроса к приложению по IPv6

1 URL / имя
Пользователь вводит адрес в браузере

2 AAAA-запись
DNS возвращает IPv6-адрес (запись AAAA)

3 IPv6-маршрут
Пакеты идут через IPv6-сеть (маршрутизаторы)

4 TCP/HTTP подключение
Устанавливается TCP-соединение к IPv6-адресу и порту

5 Ответ приложения
Приложение обрабатывает запрос и возвращает данные

Клиент / Browser



URL:
http://[2001:db8:10:20::10]:3000/

DNS



AAAA

api.company.local → 2001:db8:10:20::10

IPv6-сеть



gateway / route

Сервер

2001:db8:10:20::10



Приложение (HTTP API)



listen [::]:3000
(принимает подключения по IPv6 на порту 3000)

HTTP Response / JSON

Данные возвращаются по тому же IPv6-соединению

Что проверить программисту:

AAAA
Есть AAAA-запись в DNS

маршрут
Доступен IPv6-маршрут (gateway / route)

порт
Открыт нужный порт (например, 3000)

bind
Приложение слушает на [::]:3000

curl -6
Проверить запросом:
curl -6 http://[2001:db8:10:20::10]:3000/

Итог раздела 1

1. **IPv6 может участвовать в работе приложения даже без явной настройки разработчиком.**

2. **Главные практические зоны: DNS, URL, localhost, bind, firewall, маршруты и логи.**

3. **Для программиста IPv6 — это не теория адресации, а часть сетевого поведения приложения.**

2. Как читать IPv6-адрес и prefix

Минимум HEX, правила сокращения и /64

IPv6-адрес в одном слайде

- IPv6-адрес имеет 128 бит и записывается как восемь групп шестнадцатеричных чисел.
- Пример полного адреса: 2001:0db8:0010:0020:0000:0000:0000:0010.
- Каждая группа содержит 16 бит.
- Группы разделяются двоеточиями.
- В адресе используются цифры 0–9 и буквы a–f.
- Для учебных примеров используется диапазон 2001:db8::/32.

HEX минимум для IPv6

- Одна hex-цифра кодирует 4 бита.
- Одна группа IPv6 содержит 4 hex-цифры.
- 4 hex-цифры = 16 бит.
- 8 групп × 16 бит = 128 бит.
- Этого достаточно, чтобы понимать структуру IPv6-адреса без глубоких ручных переводов.

HEX-шпаргалка

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
8	1000	8
10	1010	A
15	1111	F

Правила сокращения IPv6

- Ведущие нули в группе можно убрать.
- 0000 можно записать как 0.
- Одну непрерывную последовательность нулевых групп можно заменить на ::.
- :: можно использовать только один раз в адресе.
- Адрес лучше записывать вместе с prefix: 2001:db8:10:20::10/64.

Примеры сокращения

Полная запись	Сокращённая запись
2001:0db8:0000:0000:0000:0000:0000:0010	2001:db8::10
fe80:0000:0000:0000:0000:0000:0000:0001	fe80::1
0000:0000:0000:0000:0000:0000:0000:0001	::1
2001:0db8:0010:0020:0000:0000:0000:0080	2001:db8:10:20::80

Prefix: что означает /64

- Prefix показывает, какая часть IPv6-адреса относится к сети.
- 2001:db8:10:20::/64 — сеть.
- 2001:db8:10:20::10/64 — адрес узла в этой сети.
- Первые 64 бита — сетевой prefix.
- Оставшиеся 64 бита — идентификатор интерфейса.
- Для пользовательских и серверных сегментов IPv6 часто используют /64.

IPv4-маска и IPv6-prefix

IPv4

- 192.168.10.20/24.
- Маска может писаться как 255.255.255.0.
- Часто считают network/broadcast.
- Адреса короче и привычнее.

IPv6

- 2001:db8:10:20::10/64.
- Обычно пишут только длину prefix.
- Broadcast в IPv6 нет.
- Важно не забывать prefix в документации.

IPv6-адрес как сеть + интерфейс

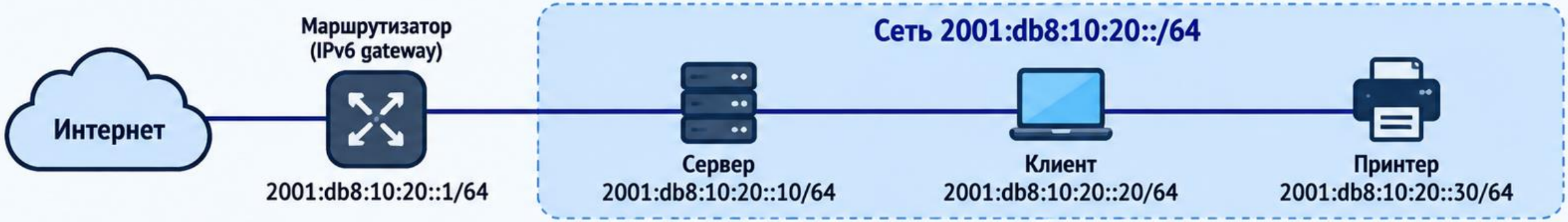
Префикс сети (network prefix)
64 бита (4 группы)

Идентификатор интерфейса (interface identifier)
64 бита (4 группы)

2001 : db8 : 10 : 20 | **0000 : 0000 : 0000 : 0010**

Префикс сети определяет, в какой IPv6-сети находится устройство.
Одинаковый префикс → одна сеть (например, /64).

Идентификатор интерфейса определяет конкретное устройство в этой сети.
У каждого устройства свой уникальный идентификатор в пределах сети.



Тот же адрес в сокращенной записи

2001:db8:10:20::10/64

Что общего в одной сети /64

- одинаковый префикс (первые 64 бита)
- могут быть разные интерфейсы
- устройства напряную видят друг друга в локальном сегменте

Примеры адресов в одной сети

2001:db8:10:20::1 — IPv6 gateway (роутер)
2001:db8:10:20::10 — сервер
2001:db8:10:20::20 — клиент
2001:db8:10:20::30 — принтер

Итог раздела 2

1. **IPv6-адрес — это 128 бит, записанные в HEX.**
2. **Ведущие нули можно убрать, а одну цепочку нулей заменить на ::.**
3. **Prefix /64 обычно отделяет сеть от идентификатора интерфейса.**
4. **Разработчику важно читать адрес и не путать IPv6-двоеточия с портом.**

3. IPv6-адреса, важные приложению

Loopback, wildcard, link-local, ULA и global

Адреса, которые программист должен узнавать

- ::1 — IPv6 loopback, аналог 127.0.0.1.
- :: — IPv6 wildcard, адрес «слушать на всех IPv6-интерфейсах».
- fe80::/10 — link-local, работает только в одном канальном сегменте.
- fc00::/7 — ULA, внутренние адреса организации.
- 2000::/3 — global unicast, потенциально маршрутизируемые адреса.

Краткая карта IPv6-адресов

Адрес / prefix	Смысл для приложения
::1	Локальный доступ только на этой машине
::	Bind на все IPv6-интерфейсы
fe80::/10	Локальный сегмент, часто нужен %interface
fc00::/7	Внутренние сервисы организации
2000::/3	Публично маршрутизируемые адреса
::/0	Default route IPv6

Loopback: 127.0.0.1 и ::1

- 127.0.0.1 — IPv4 loopback.
- ::1 — IPv6 loopback.
- Оба адреса указывают на локальную машину, но это разные IP-стеки.
- Сервер, слушающий только 127.0.0.1, может быть недоступен через ::1.
- Сервер, слушающий только ::1, может быть недоступен через 127.0.0.1.

Проверка localhost

BASH

```
getent hosts localhost
ping -c 2 127.0.0.1
ping -6 -c 2 ::1
```

Что важно

- getent показывает, как система разрешает localhost.
- ping 127.0.0.1 проверяет IPv4 loopback.
- ping -6 ::1 проверяет IPv6 loopback.
- Если приложение работает только на одном стеке, это будет важно при подключении.

Link-local и зона интерфейса

- Link-local начинается с fe80::.
- Такой адрес действует только внутри одного L2-сегмента.
- Для обращения часто нужно указать интерфейс.
- Пример: ping -6 fe80::1%ens33.
- Без %interface система может не понять, через какой интерфейс искать узел.

Проверка link-local

BASH

```
ip -6 addr show
ip -6 neigh show
ping -6 fe80::1%ens33
```

Что важно

- ip -6 addr показывает IPv6-адреса интерфейсов.
- ip -6 neigh показывает соседей IPv6.
- В команде ping вместо ens33 нужно указать реальное имя интерфейса.
- Link-local полезен для проверки локальной связи.

ТИПЫ IPv6-АДРЕСОВ

Каждый тип адреса решает свою задачу в сети и имеет свой диапазон и область действия

IPv6 использует разные типы адресов.
Администратор должен понимать область действия каждого типа и правильно применять их в инфраструктуре.

ОБОЗНАЧЕНИЯ

- Доступен в интернете
- Доступен в пределах организации
- Доступен только в пределах канала (link)
- Не маршрутизируется
- Маршрутизируется
- (A) Автоматически назначается
- (M) Назначается вручную

1 LINK-LOCAL

- Префикс: `fe80::/10`
- Область действия: Только локальный сегмент (link)
- Назначение: Связь между узлами в пределах одного канала.
- (A) Автоматически на интерфейсе



Пример: `fe80::1234:5678:9abc:def0`

2 GLOBAL UNICAST

- Префикс: `2000::/3`
- Область действия: Глобальный интернет
- Назначение: Уникальные адреса в интернет и в корпоративных сетях с глобальной маршрутизацией.
- (M) Выдаются провайдером или назначаются вручную



Пример: `2001:db8:10:1::10/64`

3 UNIQUE LOCAL (ULA)

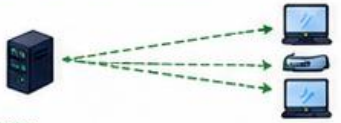
- Префикс: `fc00::/7` (чаще всего `fd00::/8`)
- Область действия: Внутри организации (не в интернете)
- Назначение: Внутренние адреса для когропративных сетей, VPN, закрытых систем.
- (M) Назначается администратором



Пример: `fd12:3456:789a::10/64`

4 MULTICAST

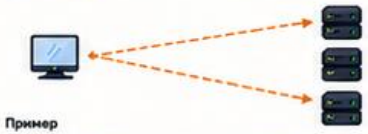
- Префикс: `ff00::/8`
- Область действия: Группа узлов (зависит от scope)
- Назначение: Передача одного потока нескольким получателям.
- (A) Используется протоколами и приложениями



Пример: `ff02::1` (All Nodes на локальной сети)

5 ANYCAST

- Префикс: В пределах Global Unicast или ULA
- Область действия: До ближайшего узла из группы
- Назначение: Доставка к ближайшему узлу, имеющему этот адрес.
- (M) Назначается вручную



Пример: `2001:db8::53` (DNS anycast)

ДОПОЛНИТЕЛЬНЫЕ СПЕЦИАЛЬНЫЕ АДРЕСА

Loopback	::1/128	Адрес обратной петли. Используется для проверки стека IPv6 на узле.	Пример: ::1
Unspecified	::/128	Неопределённый адрес. Используется источником до получения реального адреса.	Пример: ::
Discard	::/128	Адрес-«мусорная корзина». Пакеты на этот адрес должны отбрасываться.	Пример: ::
Solicited-Node	ff02::1:ff00:0/104	Автоматически используется в Neighbor Discovery для разрешения адресов узлов в MAC.	Пример: ff02::1:ffab:cdff:feef:1234
All Nodes	ff02::1	Многоадресный адрес всех узлов на локальной сети.	Пример: ff02::1
All Routers	ff02::2	Многоадресный адрес всех маршрутизаторов на локальной сети.	Пример: ff02::2

SCOPE (ОБЛАСТЬ ДЕЙСТВИЯ АДРЕСОВ)

Scope	Значение	Где используется	Примеры
Interface-local	0	Только внутри интерфейса (как правило, не используется в маршрутизации)	Служебные случаи
Link-local	2	В пределах одного канала (link)	fe80::/10, ff02::/16
Realm-local	5	В пределах домена маршрутизации	Редко используется
Admin-local	8	В пределах организации	ULA (fc00::/7)
Site-local	5 (устарел)	Концепция устарела в IPv6 (заменена на ULA)	Не используется
Global	14	Глобальный интернет	2000::/3, ff08::/16

Современные сети используют в основном: Link-local, Global, Unique Local и Multicast.

КАК АДРЕСА НАЗНАЧАЮТСЯ

Link-local	Всегда автоматически на каждом интерфейсе (EUI-64 или случайный).
Global Unicast	Выдается провайдером (SLAAC с RA) или назначается вручную.
Unique Local (ULA)	Генерируется администратором один раз для организации и используется внутри сети.
Multicast	Назначается автоматически протоколами или вручную приложениями.
Anycast	Назначается вручную нескольким узлам.

ХОРОШИЕ ПРАКТИКИ

- Используйте link-local для базовой связности и Neighbor Discovery.
- Используйте ULA для внутренних ресурсов, недоступных из интернета.
- Публикуйте в DNS только нужные глобальные IPv6-адреса (AAAA-записи).
- Документируйте префиксы и политику назначения адресов.
- Проверяйте доступность как по IPv4, так и по IPv6.

ПОЛЕЗНЫЕ КОМАНДЫ LINUX

```
ip -6 addr              # Показать IPv6-адреса интерфейсов
ip -6 route             # Показать таблицу маршрутизации IPv6
ping -6 <адрес>         # Проверка доступности по IPv6
curl -6 http://[адрес]/ # Проверка HTTP-сервиса по IPv6
ip -6 neigh             # Таблица соседей (Neighbor Discovery)
```

ПРИМЕР КОМПЛЕКТНОЙ НАСТРОЙКИ НА СЕРВЕРЕ

```
inet6 fd12:3456:789a::10/64
scope global
inet6 fe80::1234:5678:9abc:def0/64
scope link

default via fe80::1 dev eth0
metric 1024
```

ПРИМЕР ТОПОЛОГИИ



PC (Клиент)
fd12:3456:789a::20/64
GW: fe80::1

Маршрутизатор
LAN: fd12:3456:789a::1/64
WAN: 2001:db8:100::1/64

Linux-сервер
fd12:3456:789a::10/64
GW: fe80::1

ЧАСТЫЕ ОШИБКИ

- Использование адресов из разных префиксов без маршрутизации.
- Попытка доступа к link-local адресу с другого сегмента.
- Отсутствие AAAA-записей в DNS при наличии IPv6.
- Дложировка ICMPv6 или соседних сообщений firewall'ом.

Итог раздела 3

1. **Для приложений особенно важны ::1, ::, ULA и global-адреса.**

2. **Link-local требует понимания scope и интерфейса.**

3. **IPv6-адрес без контекста может быть неполным: важны prefix, scope и интерфейс.**

4. IPv6 в URL, localhost и bind

Практика backend-разработчика

IPv6-адрес в URL

- Если IPv6-адрес используется в URL вместе с портом, адрес обязательно берётся в квадратные скобки.
- Правильно: http://[2001:db8:10:20::10]:3000/
- Неправильно: http://2001:db8:10:20::10:3000/
- Скобки отделяют IPv6-двоеточия от двоеточия перед портом.
- DNS-имя в скобки не берётся: http://api.company.local:3000/.
- Эта ошибка часто встречается в конфигурациях, .env и документации API.

URL с IPv4 и IPv6

IPv4

- http://192.168.10.20:3000/
- Адрес и порт разделяются двоеточием.
- Двоеточие в адресе не используется.
- Скобки не нужны.

IPv6

- http://[2001:db8:10::20]:3000/
- В адресе уже есть двоеточия.
- Адрес заключается в квадратные скобки.
- Порт пишется после закрывающей скобки.

Проверка HTTP по IPv6

BASH

```
curl -6 -v http://[2001:db8:10:20::10]:3000/
curl -6 -v http://api.company.local:3000/
```

Что важно

- curl -6 заставляет использовать IPv6.
- -v показывает, к какому адресу выполняется подключение.
- При обращении по адресу нужны квадратные скобки.
- При обращении по имени скобки не нужны.

Bind-адрес: где слушает сервер

- Bind-адрес определяет, на каком IP-адресе приложение принимает подключения.
- 127.0.0.1 — только IPv4 loopback.
- ::1 — только IPv6 loopback.
- 0.0.0.0 — все IPv4-интерфейсы.
- :: — все IPv6-интерфейсы.
- Поведение :: относительно IPv4-mapped соединений зависит от ОС и настроек runtime.

Node.js: разные bind-адреса

```javascript
server.listen(3000, '127.0.0.1'); // только IPv4 localhost
server.listen(3000, '::1');       // только IPv6 localhost
server.listen(3000, '0.0.0.0');   // все IPv4-интерфейсы
server.listen(3000, '::');        // все IPv6-интерфейсы
```

Что важно

- Адрес bind влияет на то, откуда к приложению можно подключиться.
- Если сервер слушает только 127.0.0.1, подключение к ::1 не обязано работать.
- Если сервер слушает только ::1, подключение к 127.0.0.1 не обязано работать.
- Для публикации сервиса нужно слушать не только loopback.

Как увидеть, где слушает приложение

```bash
ss -lntp | grep 3000
```

Что важно

- ss показывает listening TCP-сокеты.
- 127.0.0.1:3000 означает IPv4 loopback.
- [::1]:3000 означает IPv6 loopback.
- [::]:3000 означает слушание на IPv6 wildcard.
- Если приложение слушает только loopback, другие машины к нему не подключатся.

Типовая ошибка localhost

- Backend запущен на 127.0.0.1:3000.
- Frontend или HTTP-клиент обращается к localhost.
- Система выбирает ::1.
- На ::1 сервер не слушает.
- Результат: connection refused.
- Решение: явно проверить bind, localhost resolution и использовать curl -4/-6.

127.0.0.1, ::1, 0.0.0.0 и ::

127.0.0.1 — IPv4 loopback



- локальный адрес IPv4
- трафик остаётся внутри этого хоста
- используется для проверки локальных сервисов

```
http://127.0.0.1:3000
```

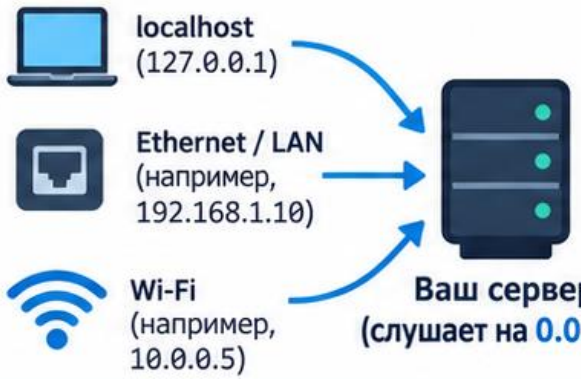
::1 — IPv6 loopback



- локальный адрес IPv6
- аналог 127.0.0.1 для IPv6
- доступ только с этого же хоста

```
http://[::1]:3000
```

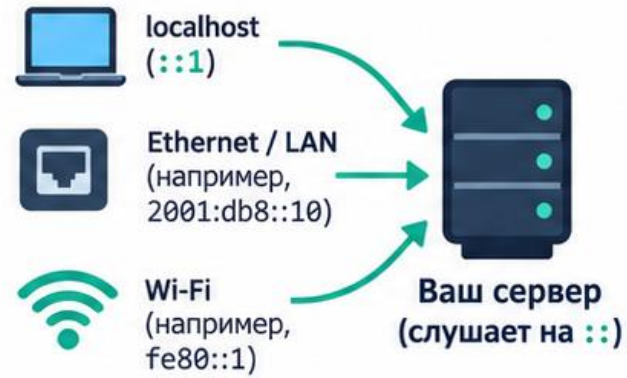
0.0.0.0 — слушать на всех IPv4-интерфейсах



- это не конкретный удалённый узел
- используется при bind/listen сервера
- сервис доступен через реальные IPv4-адреса хоста

```
app.listen(3000, '0.0.0.0')
```

:: — слушать на всех IPv6-интерфейсах



- не конкретный адрес клиента
- используется для bind/listen по IPv6
- сервис доступен через реальные IPv6-адреса хоста

```
server.listen(3000, '::')
```

Обращение к самому себе

127.0.0.1 ::1

Прослушивание всех интерфейсов

0.0.0.0 ::

127.0.0.1 и ::1 — адреса loopback.
0.0.0.0 и :: — обычно адреса привязки сервера, а не адреса назначения клиента.

Итог раздела 4

1. IPv6-адрес в URL с портом пишется в квадратных скобках.
2. localhost может привести к IPv4 или IPv6 loopback.
3. Bind-адрес определяет, где приложение реально принимает соединения.
4. Команда ss -lntp помогает отделить ошибку приложения от ошибки сети.

5. DNS A/AAAA и dual stack

Почему приложение может выбрать IPv6

A и AAAA

- A-запись связывает имя с IPv4-адресом.
- AAAA-запись связывает имя с IPv6-адресом.
- Одно имя может иметь и A, и AAAA.
- Такой режим называется dual stack.
- Клиентское приложение может выбрать IPv6, если AAAA существует.

Проверка DNS

BASH

```
dig A api.company.local
dig AAAA api.company.local
getent ahosts api.company.local
```

Что важно

- dig A проверяет IPv4-адрес.
- dig AAAA проверяет IPv6-адрес.
- getent ahosts показывает адреса так, как их видит системный resolver.
- Если AAAA есть, приложение может попытаться подключиться по IPv6.

Одно имя — A и AAAA

Одному доменному имени может соответствовать как IPv4-адрес (A), так и IPv6-адрес (AAAA). Клиент может выбрать любой из них для подключения.

1 Запрос имени

Приложение обращается к DNS для получения IP-адресов имени api.example.com.

2 DNS-ответ

DNS возвращает оба записи: A (IPv4) и AAAA (IPv6).

3 Выбор адреса

Операционная осстема и библиотека (например, getaddrinfo) могут выбрать IPv6 или IPv4 в зависимости от настроек, доступности и политики (Happy Eyeballs).

4 Подключение к серверу

Клиент устанавливает соединение с сервером по выбранному IP-адресу (IPv6 или IPv4) на нужный порт (например, 443).



Как это работает

1. Приложение запрашивает у DNS имя (A и AAAA).
2. DNS возвращает оба адреса.
3. ОС выбирает, по какому адресу подключаться (часто сначала IPv6).
4. Устанавливается TCP/TLS-соединение с выбранным адресом.

Важно для разработчика

- Если у имени есть AAAA, приложение может попытаться подключиться по IPv6.
- Если IPv6 неправильно настроен (маршрутизация, firewall), подключение может идти медленно или завершаться ошибкой.
- Для тестирования используйте принудительный выбор стека: curl -4 (IPv4) и curl -6 (IPv6).
- Сервер должен быть доступен по тем адресам, которые публикуются в DNS.

Типичная проблема

AAAA-запись существует, но IPv6 не работает.

api.example.com		
A	203.0.113.20	работает
AAAA	2001:db8:10::20	не работает

Решение: либо настроить IPv6, либо временно убрать AAAA-запись, если IPv6 недоступен.

Проблема: IPv4 работает, IPv6 сломан

- Сайт или API имеет A и AAAA.
- IPv4-путь настроен корректно.
- IPv6-адрес опубликован, но сервис не слушает IPv6 или firewall блокирует порт.
- Браузер или клиент может сначала попробовать IPv6.
- Симптом: задержки, timeout или ошибка подключения, хотя «по IPv4 всё работает».

Разделить IPv4 и IPv6

BASH

```
curl -4 -v https://api.company.local/
curl -6 -v https://api.company.local/
```

Что важно

- curl -4 принудительно проверяет IPv4.
- curl -6 принудительно проверяет IPv6.
- Если -4 работает, а -6 нет, DNS-имя уже не главный подозреваемый.
- Дальше проверяют IPv6-адрес, маршрут, firewall и listener приложения.

Когда не стоит публиковать AAAA

- Сервер не имеет рабочий IPv6-адрес.
- Приложение не слушает IPv6.
- Firewall не разрешает IPv6-трафик на нужный порт.
- TLS-сертификат или reverse proxy не проверены по IPv6.
- Мониторинг не проверяет IPv6-доступность.
- Правило: AAAA публикуется только когда IPv6-путь реально готов.

Итог раздела 5

1. Dual stack означает наличие IPv4 и IPv6 для одного сервиса.
2. AAAA-запись должна соответствовать реальной готовности приложения работать по IPv6.
3. curl -4 и curl -6 быстро разделяют проблемы IPv4 и IPv6.
4. Для программиста DNS — часть поведения клиента, а не только зона администратора.

6. Маршруты, gateway и SLAAC

Минимум сетевой логики для отладки приложения

Откуда появляется IPv6-адрес

- Адрес может быть задан статически.
- Клиент может получить prefix через Router Advertisement.
- SLAAC позволяет клиенту сформировать адрес автоматически.
- У интерфейса может быть несколько IPv6-адресов одновременно.
- Для приложения важно не только наличие адреса, но и наличие маршрута.

Что программисту нужно знать про SLAAC

- SLAAC — автоматическая настройка IPv6-адреса без классического DHCP-пула.
- Маршрутизатор сообщает prefix и параметры сети через RA.
- Клиент сам формирует IPv6-адрес.
- Серверным приложениям чаще нужен стабильный адрес или DNS-имя.
- Если есть только fe80::, маршрутизируемой IPv6-связи может не быть.

Проверка адресов и маршрутов

BASH

```
ip -6 addr show
ip -6 route show
ip -6 route show default
ip -6 route get 2001:db8:10:20::10
```

Что важно

- ip -6 addr show показывает IPv6-адреса интерфейсов.
- ip -6 route show показывает IPv6-маршруты.
- default route нужен для выхода за пределы локального сегмента.
- ip -6 route get показывает, какой путь выберет ядро для конкретного адреса.

Gateway через link-local

- В IPv6 gateway часто может быть link-local адресом.
- Пример: default via fe80::1 dev ens33.
- Интерфейс обязателен, потому что fe80:: адреса повторяются в разных сегментах.
- Если default route отсутствует, приложение может видеть локальную сеть, но не внешние IPv6-адреса.
- Если gateway недоступен, дальнейшая проверка DNS и HTTP бессмысленна.

Проверка gateway

BASH

```
ip -6 route show default
ping -6 -c 4 2001:db8:10:20::1
ping -6 -c 4 fe80::1%ens33
```

Что важно

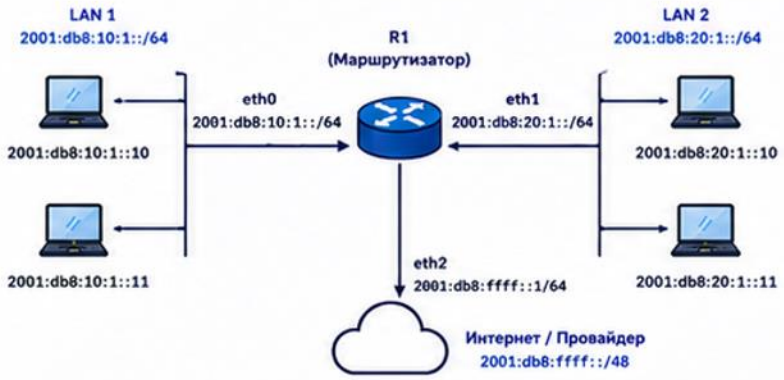
- Сначала смотрим default route.
- Если gateway глобальный — пингуем его как обычный IPv6-адрес.
- Если gateway link-local — добавляем %interface.
- Вместо ens33 укажите реальное имя интерфейса.

IPv6-МАРШРУТИЗАЦИЯ И GATEWAY

IPv6-routing управляет доставкой пакетов между сетями.
Gateway (маршрутизатор) — следующий узел на пути к другой сети.

В IPv6 маршрутизация — обязательная часть инфраструктуры.
Каждый узел сам маршрутизирует пакеты по таблице маршрутов.
Default gateway нужен для выхода во «внешние» сети и интернет.

1. ПРИМЕР СЕТИ С IPv6-МАРШРУТАМИ

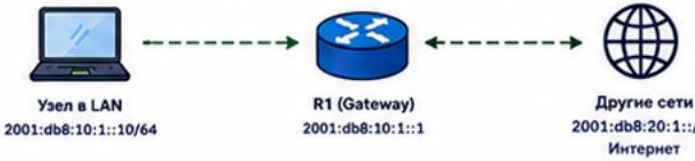


2. ТАБЛИЦА МАРШРУТОВ IPv6 (ПРИМЕР НА R1)

Destination (назначение)	Prefix Length	Next Hop (следующий узел)	Interface (интерфейс)	Тип
2001:db8:10:1::	64	::	eth0	Подключенная сеть
2001:db8:20:1::	64	::	eth1	Подключенная сеть
2001:db8:ffff::	64	::	eth2	Подключенная сеть
::/0	0	2001:db8:ffff::2	eth2	Default (по умолчанию)

- ::/0 — маршрут по умолчанию (default route).
- Если точного маршрута нет, пакет отправляется next hop для ::/0.
- Next Hop может быть link-local (fe80::/10) или глобальным адресом.

3. ЧТО ТАКОЕ GATEWAY В IPv6



Как работает gateway:

1. Хост видит, что адрес назначения не в его подсети /64.
2. Пакет отправляется на default gateway.
3. Маршрутизатор по своей таблице маршрутов выбирает выход.
4. Пакет идёт к следующему узлу и достигает цели.

Gateway — это адрес маршрутизатора в вашей локальной сети, на который отправляются пакеты во внешние сети.

4. ТИПЫ МАРШРУТОВ IPv6

Подключенный (Connected)	Сети, непосредственно подключенные к интерфейсам узла. Появляются автоматически.
Локальный (Local)	Адреса самого узла (loopback, ::1 и др.). Используются для доступа к самому себе.
Статический (Static)	Добавляются администратором вручную. Используются для удалённых сетей.
По умолчанию (Default)	Используется, если нет более конкретного маршрута. Обозначается ::/0.
Динамический (Dynamic)	Получаются по протоколам маршрутизации (OSPFv3, RIPng, BGP и др.).

5. МАРШРУТ ПО УМОЛЧАНИЮ (DEFAULT ROUTE)

Пример 1: Next Hop — глобальный адрес

```
default via 2001:db8:ffff::2 dev eth2
```

Пример 2: Next Hop — link-local адрес (указать интерфейс!)

```
default via fe80::1 dev eth2
```

Если next hop — link-local адрес, обязательно указывайте интерфейс (dev ethX), иначе маршрут будет недоступен.

6. КОМАНДЫ LINUX ДЛЯ IPv6-МАРШРУТИЗАЦИИ

Просмотр таблицы маршрутов

```
$ ip -6 route show
  или
$ ip -6 r
```

Добавить статический маршрут

```
$ sudo ip -6 route add 2001:db8:30::/64
  via 2001:db8:20:1::2 dev eth1
```

Удалить маршрут

```
$ sudo ip -6 route del 2001:db8:30::/64
  via 2001:db8:20:1::2 dev eth1
```

Добавить default route

```
$ sudo ip -6 route add default via 2001:db8:ffff::2 dev eth2
  или
$ sudo ip -6 route add default via fe80::1 dev eth2
```

7. ПРОВЕРКА МАРШРУТИЗАЦИИ

- Проверить маршруты
- Проверить доступность next hop
- Проверить связность с удалённой сетью
- Проверить выход в интернет

```
$ ip -6 route show
$ ping -6 2001:db8:20:1::10
$ ping -6 2001:db8:30::1
$ ping -6 2001:4860:4860::8888  # Google DNS IPv6
```

8. ЛОГИКА МАРШРУТИЗАЦИИ IPv6



Совпадение по prefix → самый длинный prefix → маршрут по умолчанию

9. ЧАСТЫЕ ОШИБКИ

- Не указан dev при использовании link-local адреса как next hop.
- Не настроен маршрут по умолчанию.
- Неправильный prefix (например, /64 вместо /48 у провайдера).
- Блокировка ICMPv6 — недоступна диагностика.
- Недостаточно маршрутов на промежуточных маршрутизаторах.

КРАТКИЕ ВЫВОДЫ

- IPv6-маршрутизация строится на таблице маршрутов.
- Gateway — следующий узел, на который отправляются пакеты во внешние сети.
- Правильный default route и префиксы — основа доступности IPv6.

ПОЛЕЗНЫЕ КОМАНДЫ (ШПАРГАЛКА)

```
ip -6 addr
ip -6 route show
ip -6 route add ...
ip -6 route del ...
ping -6 <адрес>
traceroute -6 <адрес>
ip -6 neigh>
sysctl -a | grep ipv6
```

СОВЕТ

Всегда проверяйте: адреса интерфейсов, таблицу маршрутов и доступность next hop.

Итог раздела 6

1. Адрес может появиться автоматически, но без маршрута приложение не подключится.
2. IPv6-маршруты нужно проверять отдельно от IPv4.
3. ip -6 route get помогает понять реальный путь пакета.
4. Link-local gateway требует указания интерфейса.

7. Диагностика IPv6-проблем приложения

От интерфейса до HTTP-клиента

Главный принцип диагностики

- Не начинайте с браузера. Проверяйте цепочку: интерфейс → адрес → маршрут → DNS → порт → приложение.
- Браузер скрывает, какой адрес и стек выбраны.
- Одна и та же ошибка может быть DNS, route, firewall или bind-проблемой.
- Каждая команда должна отвечать на один диагностический вопрос.
- Сравнивайте IPv4 и IPv6 отдельно.
- Фиксируйте шаг, на котором цепочка ломается.

ЦЕПОЧКА ДИАГНОСТИКИ IPv6

Пошаговая проверка от интерфейса до сервиса. Двигайтесь слева направо: найдите первый обрыв — он ключевой.

ПРАВИЛО ДИАГНОСТИКИ
Проверяйте каждый шаг отдельно. Если шаг не проходит — не идите дальше, пока не устраните проблему на этом шаге.

1 ИНТЕРФЕЙС
Интерфейс UP? Есть ли IPv6-адрес?

Проверка:
```
ip -6 addr
```

2 ЛИНК (L2)
Линк активен? Есть ли соседи?

Проверка:
```
ip -6 neigh
```

3 АДРЕСАЦИЯ
Адрес/префикс верный? SLAAC или статический?

Проверка:
```
ip -6 addr show dev <if>
```

4 МАРШРУТ
Есть ли маршрут к нужной сети?

Проверка:
```
ip -6 route
```

5 GATEWAY
Достижим ли default gateway?

Проверка:
```
ping -6 <gw>
или
ip -6 neigh
```

6 DNS (AAAA)
Получается ли IPv6-адрес по имени (AAAA)?

Проверка:
```
dig AAAA <домен>
или
getent ahosts <домен>
```

7 ДОСТУПНОСТЬ
Достижим ли удалённый узел по IPv6?

Проверка:
```
ping -6 <адрес>
traceroute -6 <адрес>
```

8 ПОРТ / СЕРВИС
Сервис слушает IPv6? Порт открыт?

Проверка:
```
ss -ltnp | grep :<порт>
curl -6 http://[адрес]/
```

9 ФАЙРВОЛ
Не блокирует ли firewall трафик?

Проверка:
```
sudo ip6tables -L -n -v
или firewall-cmd --list-all
```

10 ПРИЛОЖЕНИЕ
Работает ли приложение корректно?

Проверка:
```
логи приложения
и системные логи
```

✅ Все шаги пройдены — IPv6 работает

⚠ Не работает на каком-то шаге — вернитесь и исправьте проблему

ПОДРОБНО ПО ШАГАМ: КОМАНДЫ И ОЖИДАЕМЫЙ РЕЗУЛЬТАТ

1 Интерфейс
Команда:
```
ip -6 addr
```
Ожидаемый результат:
```
inet6 2001:db8:10::20/64
scope global dynamic
valid_lft ... preferred_lft ...
```
Проблемы:
- interface DOWN
- нет IPv6-адреса

2 Линк (L2)
Команда:
```
ip -6 neigh
```
Ожидаемый результат:
```
2001:db8:10::1 dev eth0
lladdr aa:bb:cc:dd:ee:ff
REACHABLE
```
Проблемы:
- нет записей
- состояние STALE/FAILED

3 Адресация
Команда:
```
ip -6 addr show dev eth0
```
Ожидаемый результат:
```
inet6 2001:db8:10::20/64
scope global dynamic
```
Проблемы:
- неверный /prefix
- адрес не глобальный

4 Маршрут
Команда:
```
ip -6 route
```
Ожидаемый результат:
```
default via 2001:db8:10::1
dev eth0 proto ra metric 1024
```
Проблемы:
- нет default route
- неверный next hop

5 Gateway
Команда:
```
ping -6 2001:db8:10::1
```
Ожидаемый результат:
```
64 bytes from 2001:db8:10::1:
icmp_seq=1 ttl=64 time=1.1 ms
```
Проблемы:
- timeout / unreachable
- неверный адрес GW

6 DNS (AAAA)
Команда:
```
dig AAAA example.com
```
Ожидаемый результат:
```
example.com. 300 IN AAAA
2001:db8:abcd::1234
```
Проблемы:
- NOERROR, но нет AAAA
- SERVFAIL / NXDOMAIN

7 Доступность
Команда:
```
ping -6 2001:db8:abcd::1234
traceroute -6 2001:db8:abcd::1234
```
Ожидаемый результат:
ответы от узла,
трассировка без обрывов

Проблемы:
- timeout на хопах
- Policy / ACL / MTU

8 Порт / сервис
Команда:
```
ss -ltnp | grep :80
curl -6 http://[2001:db8:abcd::1234]/
```
Ожидаемый результат:
```
LISTEN 0 128 [::]:80 ..
HTTP/1.1 200 OK
```
Проблемы:
- сервис слушает только IPv4
- connection refused

9 Файрвол
Команда:
```
sudo ip6tables -L -n -v
```
Ожидаемый результат:
```
ACCEPT tcp -- ::/0 ::/0
tcp dpt:80
```
Проблемы:
- DROP/REJECT
- счётчики растут

10 Приложение
Команда:
```
journalctl -u <service> -n 50
```
или
логи приложения

Ожидаемый результат:
ошибок нет,
сервис отвечает по IPv6

Проблемы:
ошибки в логах,
неверная привязка к IPv6

БЫСТРЫЙ ЧЕК-ЛИСТ

- ✅ Интерфейс UP и имеет глобальный IPv6-адрес (/64).
- ✅ Есть default маршрут через правильный gateway.
- ✅ Gateway отвечает на ping -6.
- ✅ DNS возвращает AAAA-запись.
- ✅ Удалённый узел доступен по ping -6.
- ✅ Сервис слушает на IPv6 и порт открыт.
- ✅ Firewall разрешает IPv6-трафик.
- ✅ В логах приложения нет ошибок.

ПРИМЕР ДИАГНОСТИКИ: ВЕБ-СЕРВЕР НЕ ДОСТУПЕН ПО IPv6

```
1. ip -6 addr show eth0              -> OK: 2001:db8:10::20/64
2. ip -6 route                       -> OK: default via 2001:db8:10::1
3. ping -6 2001:db8:10::1            -> OK: replies
4. dig AAAA example.com              -> OK: 2001:db8:abcd::1234
5. ying -6 2001:db8:abcd::1234       -> OK: replies
6. curl -6 http://[2001:db8:abcd::1234]/  -> Failed: Connection refused
7. ss  -ltnp | grep :80          -> LISTEN  0  128  0.0.0.0:80 (IPv4 only)
```

⚠ Вывод: сервис слушает только IPv4 → настроить слушание на [::]:80

ПОЛЕЗНЫЕ КОМАНДЫ IPv6

Показать адреса:	`ip -6 addr`
Показать маршруты:	`ip -6 route`
Показать соседей:	`ip -6 neigh`
Проверить имя (AAAA):	`dig AAAA <домен>`
Проверить доступность:	`ping -6 <адрес>`
Трассировка:	`traceroute -6 <адрес>`
Проверить порт:	`ss -ltnp | grep :<порт>`
HTTP по IPv6:	`curl -6 http://[адрес]/`

ЧАСТЫЕ ПРИЧИНЫ ПРОБЛЕМ

- Нет глобального IPv6-адреса на интерфейсе.
- Нет default маршрута или неверный next hop.
- DNS не возвращает AAAA-записи.
- Firewall блокирует IPv6 (icmpv6 или tcp/udp).
- Сервис слушает только IPv4 (0.0.0.0), а не [::].
- MTU/фрагментация блокируется по пути.

СОВЕТ: всегда фиксируйте вывод команд и шаг, на котором обнаружена проблема. Это экономит время при повторной диагностике и эскалации.

IPv6 работает, когда верна вся цепочка, а не только отдельный узел.

Алгоритм диагностики

1	**Шаг 1**	Проверить интерфейс: ip link show.
2	**Шаг 2**	Проверить IPv6-адрес: ip -6 addr show.
3	**Шаг 3**	Проверить маршрут: ip -6 route show.
4	**Шаг 4**	Проверить gateway: ping -6 адрес_gateway.
5	**Шаг 5**	Проверить DNS: dig AAAA имя.
6	**Шаг 6**	Проверить выбор адреса: getent ahosts имя.
7	**Шаг 7**	Проверить listener: ss -lntp.
8	**Шаг 8**	Проверить приложение: curl -6 -v URL.
9	**Шаг 9**	Сравнить с IPv4: curl -4 -v URL.
10	**Шаг 10**	Сделать вывод: где именно обрыв.

Мини-чеклист команд

BASH

```
ip -6 addr show
ip -6 route show
dig AAAA api.company.local
getent ahosts api.company.local
ss -lntp | grep 3000
curl -6 -v http://api.company.local:3000/
curl -4 -v http://api.company.local:3000/
```

Что важно

- Первые две команды проверяют локальную IPv6-сетевую часть.
- dig и getent проверяют DNS и системный resolver.
- ss проверяет, слушает ли приложение нужный порт.
- curl -6 проверяет именно IPv6-подключение.
- curl -4 нужен для сравнения с IPv4.

Симптомы и вероятная причина

Симптом	Вероятная причина
curl -4 работает, curl -6 нет	IPv6 route, firewall, listener или AAAA
dig AAAA возвращает адрес, curl -6 не работает	DNS уже не главный подозреваемый
ss показывает 127.0.0.1:3000	Приложение слушает только IPv4 loopback
ss показывает [::1]:3000	Приложение слушает только IPv6 loopback
В URL IPv6 без []	Неверный синтаксис URL
Есть только fe80::	Нет global/ULA для маршрутизируемой связи
ping gateway не работает	Дальше проверять HTTP бессмысленно

Что писать в отчёт разработчика

- IPv6-адрес клиента и сервера с prefix.
- AAAA-запись имени сервиса.
- Результат curl -4 и curl -6.
- Вывод ss -lntp для порта приложения.
- Фактический URL, включая квадратные скобки для IPv6-адреса.
- Шаг диагностики, на котором обнаружен обрыв.
- Исправление: DNS, bind, route, firewall или конфигурация приложения.

Итоги лекции

1. IPv6 для программиста — это DNS, URL, bind, localhost, маршруты и диагностика.
2. IPv6-адрес в URL с портом требует квадратных скобок.
3. localhost может выбрать ::1, поэтому bind-адрес важен.
4. AAAA-запись должна публиковаться только при рабочем IPv6-пути.
5. curl -4 и curl -6 помогают быстро разделить IPv4 и IPv6.
6. Диагностика должна идти по цепочке: адрес → маршрут → DNS → порт → приложение.

Контрольные вопросы

1. Почему IPv6 может повлиять на приложение, даже если разработчик его явно не настраивал?
2. Зачем IPv6-адрес в URL берётся в квадратные скобки?
3. Чем 127.0.0.1 отличается от ::1?
4. Что означает bind на 0.0.0.0 и на ::?
5. Что показывает команда ss -lntp?
6. Чем DNS-запись A отличается от AAAA?
7. Зачем сравнивать curl -4 и curl -6?
8. Что означает prefix /64 в IPv6-адресе?
9. Почему link-local адресу часто нужен %interface?
10. Что проверить, если dig AAAA работает, а curl -6 не подключается?
11. Почему AAAA-запись нельзя публиковать без проверки приложения по IPv6?
12. Какая цепочка диагностики помогает найти проблему IPv6-подключения?